FRIDA

Frida LAB 11

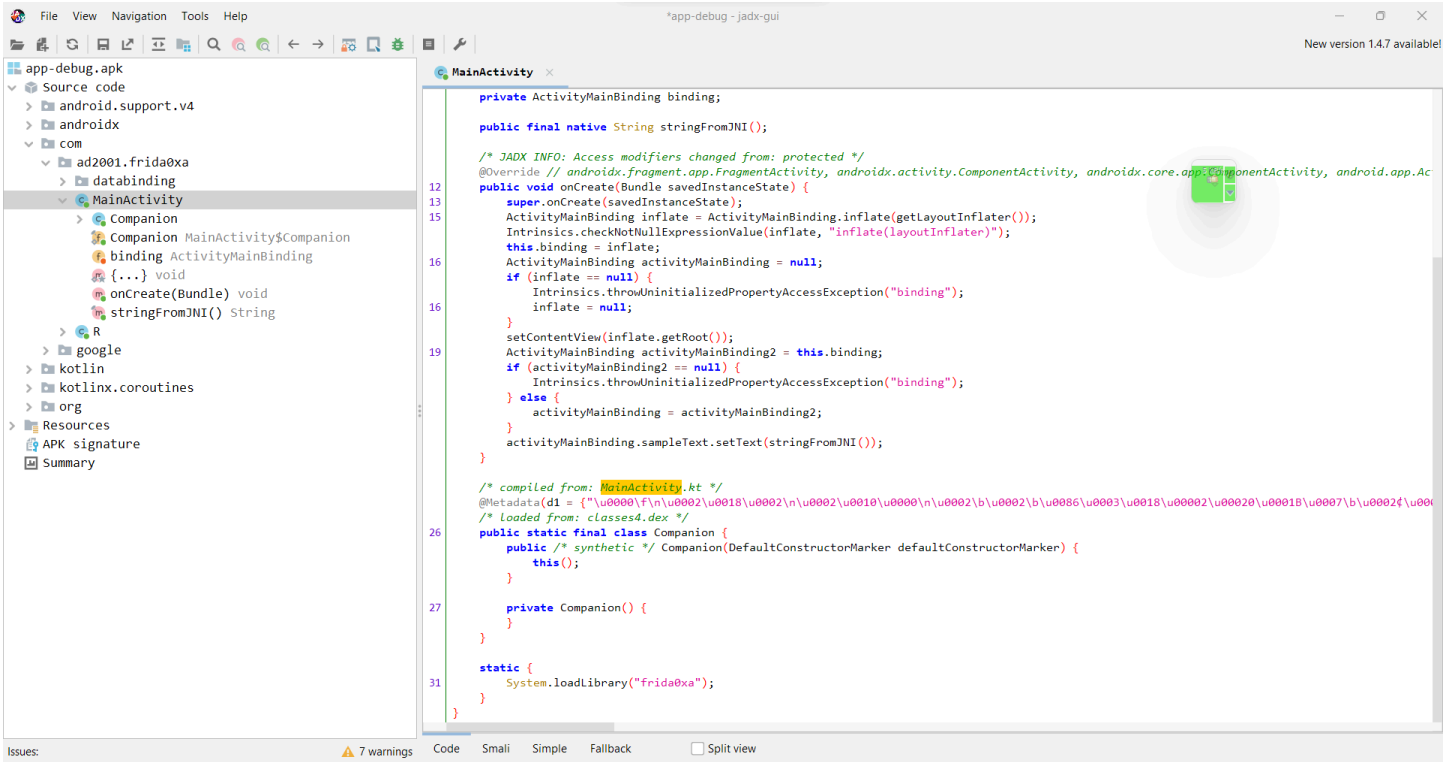
Objective:

Objective

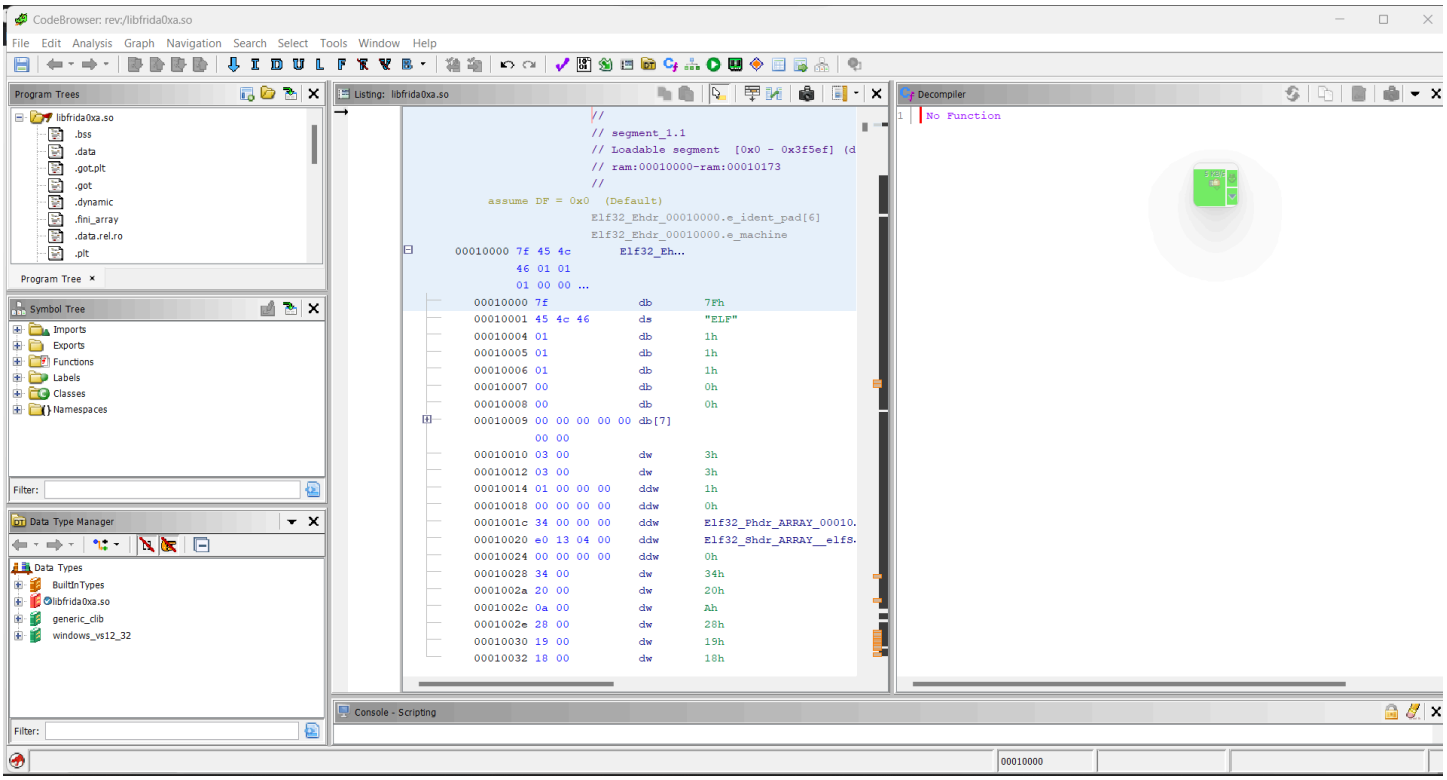
The objective of this challenge is to learn runtime patching of native instructions using Frida. The application contains a native function where a conditional jump prevents the execution of the flag-decoding code.

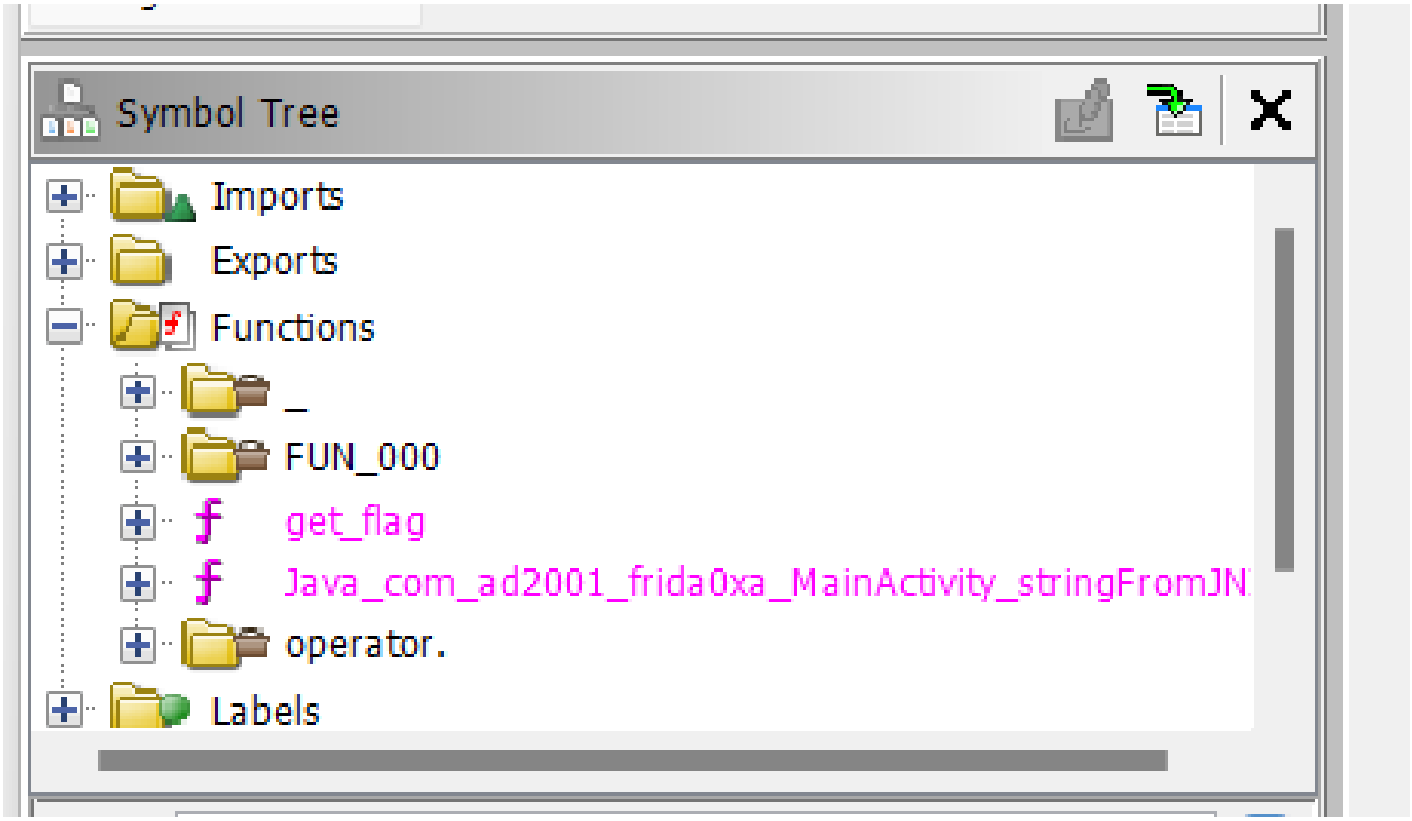
Steps

1. **Analyze the APK**
   - **Open the APK in JADX.**
   - **Identify the native function getFlag() and the loaded library libfrida0xb.so.**



1. **Reverse Engineer the Native Library**
   - **Open libfrida0xb.so in Ghidra.**
   - **Locate the conditional jump instruction (JNZ on x86 or B.NE on ARM64) that skips the flag-decoding logic.**





-

Calculate the Runtime Address

- **Obtain the library base address using Frida:**
- **Module.getBaseAddress("libfrida0xb.so")**
- **Add the instruction offset to calculate the target address.**

Decompile: get_flag - (libfridaUxa.so)

```
1
2  /* get_flag(int, int) */
3
4  void get_flag(int param_1,int param_2)
5
6  {
7    uint uVar1;
8    int in_GS_OFFSET;
9    uint local_30;
10   char local_20 [19];
11   undefined local_d;
12   int local_c;
13
14   local_c = *(int *)(in_GS_OFFSET + 0x14);
15   if (param_1 + param_2 == 3) {
16     local_30 = 0;
17     while( true ) {
18       uVar1 = __strlen_chk("FPE>9q8A>BK-)20A-#Y",0xffffffff);
19       if (uVar1 <= local_30) break;
20       local_20[local_30] = "FPE>9q8A>BK-)20A-#Y"[local_30] + (char)(local_30 << 1);
21       local_30 = local_30 + 1;
22     }
23     local_d = 0;
24     __android_log_print(3,&DAT_0001bf62,"Decrypted Flag: %s",local_20);
25   }
26   if (*(int *)(in_GS_OFFSET + 0x14) == local_c) {
27     return;
28   }
29                     /* WARNING: Subroutine does not return */
30   __stack_chk_fail();
31 }
32
```

Modify Memory Protection

- **Make the target memory writable before patching:**
- **Memory.protect(address, 0x1000, "rwx");**

```
00115240 a8 c3 5d b8     ldur        w8,[x29, #local_34]
00115244 08 e5 14 71     subs        w8,w8,#0x539
00115248 21 07 00 54     b.ne        LAB_0011532c
0011524c 01 00 00 14     b           LAB_00115250

                     LAB_00115250                                    XREF[1]:     0011524c(j)
```

Execute the Application

- **Run the Frida script and click the button in the application.**

```
Thank you for using Frida!
PS C:\Users\ajind> frida -U -f com.ad2001.frida0xa
     ____
    / _  |   Frida 16.1.5 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `com.ad2001.frida0xa`. Resuming main thread!
[Android Emulator 5554::com.ad2001.frida0xa ]->
```

- **The patched code executes the previously unreachable flag-decoding logic.**

```
PS C:\Users\ajind> frida -U -f com.ad2001.frida0xa
     ____
    / _  |   Frida 16.1.5 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Spawned `com.ad2001.frida0xa`. Resuming main thread!
[Android Emulator 5554::com.ad2001.frida0xa ]-> var adr = Module.findBaseAddress("libfrida0xa.so").add(0x18BB0) //Address of the get_flag() function
var get_flag_ptr = new NativePointer(adr);
const get_flag = new NativeFunction(get_flag_ptr, 'void', ['int', 'int']);
get_flag(1,2);
[Android Emulator 5554::com.ad2001.frida0xa ]->
```